

Experiment - 09

Student Name: Priyanshu
Branch: CSE - AIML
Semester: 5
Subject Name: ADBMS

UID: 24BAI70700
Section/Group: 24AIT_KRG-1_B
Date of Performance: 14/9/26
Subject Code: 24CSP-310

(A)

An organization stores employee details in the employee_info table. Create a PostgreSQL stored procedure that accepts an employee's gender as an input parameter and calculates the total number of employees belonging to that gender. The calculated count should be returned through an OUT parameter and should also be displayed as a message using RAISE NOTICE.

For example, if Male is passed to the procedure, it should count all employees whose gender is Male.

Column Name	Data Type	Description
id	SERIAL	Unique employee ID
name	VARCHAR(50)	Employee name
gender	VARCHAR(10)	Gender of employee
salary	NUMERIC(10,2)	Employee salary
city	VARCHAR(50)	Employee city

Solution:

```
CREATE OR REPLACE PROCEDURE SP_GET_GENDER_COUNT(IN
P_GENDER VARCHAR(20), OUT P_TOTAL_COUNT INT)
LANGUAGE PLPGSQL
AS
$$
    BEGIN
        SELECT COUNT(ID)
        INTO P_TOTAL_COUNT
        FROM EMPLOYEE_INFO
        WHERE GENDER = P_GENDER;
```

```

        RAISE NOTICE 'TOTAL EMPLOYEES WITH GENDER %: %',
P_GENDER, P_TOTAL_COUNT;
    END;
$$

```

```
CALL SP_GET_GENDER_COUNT('Female', NULL);
```

(B)

An e-commerce company maintains product stock information in the products table and stores completed orders in the sales table. Create a PostgreSQL stored procedure that accepts a product name and required quantity as input parameters and processes the purchase only when sufficient stock is available.

The procedure should verify the available quantity of the requested product. If sufficient stock exists, it should retrieve the product code and price, calculate the total sale amount, insert a new order into the sales table, reduce the available stock, and increase the total quantity sold. If sufficient stock is not available, the procedure should not modify any data and should display an appropriate notification message.

The total sale amount should be calculated as:

Sale Price = Product Price × Quantity Ordered

Tables:

Column Name	Data Type	Description
product_code	VARCHAR(10)	Unique product code
product_name	VARCHAR(100)	Product name
price	NUMERIC(10,2)	Price of one unit
quantity_remaining	INT	Current available stock
quantity_sold	INT	Total quantity sold

Column Name	Data Type	Description
order_id	SERIAL	Unique order ID
order_date	DATE	Date on which order was placed
product_code	VARCHAR(10)	Product purchased
quantity_order ed	INT	Quantity purchased
sale_price	NUMERIC(10,2)	Total order amount

Solution:

```

CREATE OR REPLACE PROCEDURE PR_BUY_PRODUCTS(IN
P_PRODUCT_NAME VARCHAR(20), IN P_QUANTITY INT)
LANGUAGE PLPGSQL
AS
$$

```

```
DECLARE
    V_PRODUCT_CODE VARCHAR;
    V_PRICE FLOAT;
    V_COUNT INT;
BEGIN
    SELECT COUNT(*)
    INTO V_COUNT
    FROM PRODUCTS
    WHERE PRODUCT_NAME=P_PRODUCT_NAME
    AND QUANTITY_REMAINING >= P_QUANTITY;

    IF V_COUNT>0 THEN
        SELECT PRODUCT_CODE, PRICE
        INTO V_PRODUCT_CODE, V_PRICE
        FROM PRODUCTS
        WHERE PRODUCT_NAME=P_PRODUCT_NAME;

        INSERT INTO SALES (ORDER_DATE, PRODUCT_CODE,
        QUANTITY_ORDERED, SALE_PRICE)
        VALUES (CURRENT_DATE, V_PRODUCT_CODE, P_QUANTITY,
        (V_PRICE*P_QUANTITY));

        UPDATE PRODUCTS
        SET QUANTITY_REMAINING = QUANTITY_REMAINING -
        P_QUANTITY,
        QUANTITY_SOLD = QUANTITY_SOLD + P_QUANTITY
        WHERE PRODUCT_CODE = V_PRODUCT_CODE;

        RAISE NOTICE 'PRODUCT SOLD..! Order placed
        successfully for % unit(s) of %.', P_QUANTITY, P_PRODUCT_NAME;

    ELSE
        RAISE NOTICE 'INSUFFICIENT QUANTITY..! Order cannot be processed
        for % unit(s) of %.', P_QUANTITY, P_PRODUCT_NAME;
    END IF;
END;
$$

CALL PR_BUY_PRODUCTS('Samsung Galaxy S23 Ultra', 1);
SELECT * FROM PRODUCTS;
```